

Tamaño del disco y estrategia de planificación de acceso

1 Tamaño del disco

Paso 1. Determinar las superficies útiles

El disco tiene 6 platos y cada plato tiene 2 superficies magnéticas, excepto el primero y el último, que solo aportan una cada uno (la cara superior del primer plato y la inferior del último no se usan):

$$\text{superficies} = 6 \times 2 - 2 = 10$$

Esto coincide con las 10 cabezas de lectura/escritura del enunciado: una cabeza por superficie.

Paso 2. Calcular el total de sectores

El total de sectores es el producto de cabezas, cilindros y sectores por pista:

$$\text{sectores} = 10 \times 128 \times 64 = 81,920 \text{ sectores}$$

Paso 3. Convertir a bytes

Cada sector guarda 512 bytes:

$$\text{tamaño} = 81,920 \times 512 = 41,943,040 \text{ bytes} = 40 \text{ MB}$$

Paso 4. Expresar en clusters

Con clusters de 4 Kb (4096 bytes, es decir 8 sectores cada uno):

$$\text{clusters} = 41,943,040 / 4,096 = 10,240 \text{ clusters}$$

2 Estrategia de planificación para minimizar los movimientos mecánicos

Paso 1. Identificar qué es mecánico y qué no

En un disco los movimientos mecánicos son dos: el desplazamiento del brazo entre cilindros (seek) y la rotación que hay que esperar para que el sector pase debajo de la cabeza (latencia rotacional). El cambio de una cabeza a otra, en cambio, es electrónico y no cuesta movimiento. La estrategia aprovecha esto en los pasos siguientes.

Paso 2. Mover el brazo con SCAN (algoritmo del elevador)

El brazo barre los cilindros en un solo sentido atendiendo todas las solicitudes que va encontrando y, al llegar al final, regresa barriendo en sentido contrario. Así se eliminan

los viajes de ida y vuelta que produce atender en orden de llegada (FIFO) y, a diferencia de SSTF, ninguna solicitud de los cilindros extremos queda postergada indefinidamente.

Paso 3. Agotar el cilindro antes de mover el brazo

Antes de desplazar el brazo se atienden todas las solicitudes pendientes del cilindro actual, sin importar en qué cabeza estén. Como el cilindro agrupa las 10 pistas que quedan bajo las cabezas sin mover el brazo, y conmutar de cabeza es electrónico, todo lo que esté en el mismo cilindro sale sin costo mecánico adicional de seek.

Paso 4. Dentro del cilindro, agrupar por sector para explotar el buffer del firmware

Cuando se lee un sector se activan las 10 cabezas a la vez y el firmware guarda ese sector de todas las pistas del cilindro en su buffer interno. Entonces, si el sistema operativo ordena consecutivas las solicitudes que piden el mismo sector en distintas cabezas, solo la primera hace la lectura física (y paga la rotación); las demás se sirven directamente del buffer y solo pagan la transferencia a memoria de 1/8 de segundo por sector, sin ningún movimiento mecánico. Además, recorrer los grupos en orden creciente de sector reduce la espera rotacional dentro del cilindro, que en este disco no es despreciable:

$$t_{vuelta} = 64 \text{ sectores} / 32 \text{ sectores/s} = \mathbf{2 \text{ s por revolución}}$$

Paso 5. Orden de atención resultante

(cilindro en modo elevador → sector → cabeza)

Vale aclarar que el tiempo de transferencia a memoria es un costo fijo de 1/8 de segundo por cada sector solicitado y no depende del orden en que se atienda la cola, así que lo único que la planificación puede reducir, y con esta estrategia reduce, es el trabajo mecánico.

Paso 6. Ejemplo de la estrategia en código

Para ejemplificar, la estrategia se puede expresar en C++ como una función que recibe la cola de solicitudes pendientes y la posición actual del brazo, y devuelve la cola en el orden en que el sistema operativo debería atenderla. Primero se ordena por (cilindro, sector, cabeza), que es justo el criterio de los pasos anteriores, y luego se parte en el barrido de subida y el de bajada para implementar el SCAN:

```

struct Solicitud { int cil, cab, sec; };

vector<Solicitud> planificar(vector<Solicitud> q, int cilActual) {
    // criterio: cilindro, luego sector, luego cabeza
    sort(q.begin(), q.end(), [](const Solicitud& a, const Solicitud& b) {
        if (a.cil != b.cil) return a.cil < b.cil;
        if (a.sec != b.sec) return a.sec < b.sec;
        return a.cab < b.cab;
    });
    // SCAN: primero el barrido hacia arriba, despues el de bajada
    vector<Solicitud> subida, bajada;
    for (const Solicitud& s : q)
        (s.cil >= cilActual ? subida : bajada).push_back(s);
    reverse(bajada.begin(), bajada.end());
    subida.insert(subida.end(), bajada.begin(), bajada.end());
    return subida;
}

```

Con este orden, las solicitudes del mismo cilindro quedan juntas (el brazo no se mueve entre ellas) y, dentro del cilindro, las del mismo sector en distintas cabezas quedan consecutivas: la primera hace la lectura física y las demás se sirven del buffer interno del disco, pagando solo el 1/8 de segundo de transferencia.